



Assignment 2 (30 pts)

Due March 20, 2025

Q1 (6pts) The following problems require you to investigate the nature of critical points in unconstrained optimization problems.¹

- (a) (1.5pt) $f(x, y) = x^3 - 3x + y^2$
- Find all critical points of f .
 - For each critical point, determine whether the first-order necessary conditions and second-order sufficient conditions are satisfied.
 - Classify each critical point (local minimum, local maximum, saddle point, or other). Justify your answers geometrically.
- (b) (1.5pt) $f(x, y) = x^4 + y^2$
- Identify all critical points of f .
 - At each critical point, analyze whether the standard optimality conditions (first and second-order) provide conclusive information about the nature of the point.
 - If the standard optimality conditions are inconclusive at any point, develop alternative approaches to determine the true nature of these points.
 - Is this function bounded below? Bounded above? Explain how your answers to parts (i)-(iii) relate to these properties.
- (c) (1.5pts) $f(x, y) = (y - x^2)^2$
- Find the set of all critical points of f .
 - What challenges arise when applying the standard optimality conditions to this function?
 - For points that satisfy the first-order necessary condition, carefully analyze the Hessian matrix. What do you observe?

¹**Guidelines:**

- For each problem, begin with a systematic application of the first and second-order optimality conditions. When the first and second-order conditions don't provide conclusive answers, develop alternative analytical or geometric approaches.
- Explain your reasoning clearly and include relevant visualizations or sketches where helpful to support your analysis.

- iv. Develop a geometric interpretation of the function and its critical points. What is the global behavior of this function? Does it have a minimum value? If so, is this minimum achieved at isolated points or along a set?
- (d) **(1.5pts)** $f(x, y) = x^3 + xy^3$
- i. Find all critical points of f .
 - ii. At each critical point, compute and analyze the Hessian matrix. What information does it provide?
 - iii. If the Hessian analysis is inconclusive at any critical point, develop and apply higher-order methods to determine the nature of these points.
 - iv. Consider the behavior of f along various paths through its critical points (e.g., along coordinate axes, diagonal lines). What insights does this approach provide about the function's local and global behavior?

Q2 (5pts) Steepest descent method for convex quadratics

(a) Formulation and implementation (2.5pts):

- Derive the exact line search formula for steepest descent when minimizing convex quadratics of the form $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b}$, where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is SPD and $\mathbf{b} \in \mathbb{R}^n$.
- Implement the steepest descent algorithm using exact line search.
- Run for exactly 1000 iterations (no early stopping)
- Record the objective function value and gradient norm at each iteration²

(b) Performance analysis (2.5pts):

- Test your implementation on the provided case in `ConvexQuadratic.mat`, which contains a sparse 341×341 SPD matrix $\mathbf{A}$ and the corresponding vector $\mathbf{b}$. Use 10 random initial guesses with entries uniformly distributed in $[-10, +10]^n$.
- Create plots displaying the convergence of the objective function value and the gradient norm³ for each initial guess.
- Include at least three representative convergence plots in your report.
- Calculate and report the lowest gradient norm achieved within 1000 iterations across all trials

Q3 (8pts) Implement a quasi-Newton algorithm with the BFGS Hessian approximation scheme for solving unconstrained optimization problems.⁴

²**Note:** Save your implementation and results for comparison with the Barzilai-Borwein method in Q4. For a fair comparison, use the same set of initial points and the same number of iterations (1000) for both methods.

³Always use semilogy plots for the gradient norm.

⁴**Guidelines:** In your report, include a pseudocode describing your BFGS implementation, clear visualizations of the comparative performance, discussion of results including any numerical challenges encountered.

(a) **Implementation (4pts):** Here are some implementation guidelines:

- Initialize Hessian approximation with an identity matrix
- Back-tracking line search based on Armijo's sufficient decrease condition (use $\mu_1 = 10^{-4}$ and $\rho = 0.5$ as default parameters)
- Appropriate termination criteria based on the gradient norm, maximum number of iterations, and relative change in the objective function value
- Safeguards against potential numerical issues

(b) **Performance Analysis (4pts):**

- Test your implementation on the Rosenbrock test function (with $n = 2$ and $n = 5$) for 10 random initial guesses uniformly distributed in $[-5, +5]^n$
- Compare against the steepest descent algorithm with the same back-tracking line search parameters
- Create plots showing the convergence of the objective function value and the gradient norm as a function of the total number of function evaluations.
- Compute and report average iterations to convergence, average function evaluations, and success rate for both methods

Q4 (6pts) In 1988, Barzilai and Borwein (BB) published a gradient-based optimization algorithm⁵ that exhibits unusual but potentially efficient convergence behavior.⁶

Similar to the steepest descent method, the BB algorithm updates iterates as: $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)$. However, the step length α_k is derived through a different approach, designed to approximate the action of the inverse Hessian without explicitly computing it:

$$\alpha_k = \frac{\mathbf{s}_k^T \mathbf{s}_k}{\mathbf{s}_k^T \mathbf{y}_k},$$

where $\mathbf{s}_k = \mathbf{x}_k - \mathbf{x}_{k-1}$ and $\mathbf{y}_k = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1})$.

(a) **Implementation (2pts):** Implement the BB algorithm for minimizing the convex quadratic function $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b}$, where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is SPD and $\mathbf{b} \in \mathbb{R}^n$.

- In the first iteration, use the steepest descent method with exact line search. In subsequent iterations, use the BB step size formula.
- Run for 1000 iterations (no early stopping)

⁵J. Barzilai and J. Borwein, "Two-point step size gradient method," *IMA Journal of Numerical Analysis*, Vol. 8, 1988, pp. 141–148

⁶The key insight from this exercise is understanding that strict monotonic decrease in the objective function is not always necessary for convergence, and sometimes allowing non-monotonic behavior can lead to more efficient algorithms.

- Test on the provided case in `ConvexQuadratic.mat`, which contains a sparse 341×341 SPD matrix $\mathbf{A}$ and the corresponding vector $\mathbf{b}$.
 - Run your code on 10 random initial guesses with entries uniformly distributed in $[-10, +10]^n$.
- (b) **Analysis of non-monotonic convergence (4pts):** Carefully study and document the convergence behavior of the BB algorithm.
- Create plots showing the convergence of the objective function value and the gradient norm for at least 3 representative initial guesses
 - Compare the performance of the BB algorithm to your steepest descent method implementation in Q2 using the same initial guesses
 - For all 10 initial guesses, calculate and report statistics on: (i) total iterations to convergence, (ii) the percentage of iterations where $f(\mathbf{x}_{k+1}) > f(\mathbf{x}_k)$, and (iii) the maximum observed increase in the objective function value between consecutive iterations
 - Discuss whether the non-monotonic behavior varies significantly across different initial points, and if so, what patterns you observe

Q5 (5pts) Consider a gradient-based optimization algorithm for solving the unconstrained minimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}),$$

wherein the search direction is given by $\mathbf{p}_k = -\mathbf{g}_k = -\nabla f(\mathbf{x}_k)$ when the iteration counter $k = 0$. When the iteration counter $k = 1$, the search direction is set to $\mathbf{p}_k = -\mathbf{g}_k + \mathbf{p}_{k-1}(\mathbf{g}_k^T \mathbf{g}_k) / (\mathbf{g}_{k-1}^T \mathbf{g}_{k-1})$. For subsequent iterations ($k > 1$), the search direction is computed as:

$$\mathbf{p}_k = -\alpha_k \mathbf{g}_k + \beta_k \mathbf{p}_{k-1} + \gamma_k \mathbf{p}_{k-2}, \quad k > 1,$$

where $\alpha_k, \beta_k, \gamma_k \in \mathbb{R}$.

Derive an approach to estimate the parameters α_k , β_k and γ_k such that the search direction is close (with respect to some appropriate norm that you are free to choose) to the Newton search direction. Your approach must not assume that any of the parameters are zero and the computational complexity must be $\mathcal{O}(n^2)$.

Clearly show all working steps in your derivation. Define the norm you are using to measure the closeness of the search directions, state any assumptions you are making, and evaluate the computational complexity of your algorithm.

Bonus (5pts) For the optimization algorithm described in Q5, derive an approach to estimate the parameters α_k , β_k and γ_k in the search direction using *only* the first-order derivatives of f . Your approach must not use finite difference or complex step approximations to the second-order derivatives of f . In addition, your approach must not assume that any of the parameters are zero. Clearly show all working steps in your derivation and explain if your method will always provide a valid direction of descent.

Submission guidelines:

- Your assignment must be submitted in **pdf** format to Quercus. Reports submitted in other formats such as Microsoft Word are not acceptable.
- You are permitted to use Python for this assignment. Please mention the Python version you are using in your report.
- Submit well-documented MATLAB/Python codes archived in **tar**, **zip** or **gzip** format along with input files used to generate results. Do not use proprietary data compression formats such as **rar**. Please verify the integrity of your **tar/zip/gzip** file before uploading.
- Include a **README** file that describes how the codes must be run.
- Clearly explain all working steps in your answers.
- All submissions must strictly comply with the AER1415 Collaboration Policy posted on Quercus.